

计算机网络课程报告

wireshark 抓包分析



学 号 2351520

姓 名 毛星博

专 业 计算机科学与技术

课程名称 计算机网络

授课老师 陆有军

一、实验目的

1.掌握 Wireshark 抓包工具的基本使用方法

- 熟悉 Wireshark 软件的安装与网络接口（Interface）的选择。
- 掌握显示过滤器（Display Filter）的语法规则（如 ip.addr, tcp, http 等），以便从海量数据中精准筛选目标报文。
- 学会利用“追踪流”（Follow Stream）功能重组完整的数据传输会话。

2.理解并分析常见网络协议的结构与通信过程

- 基于 OSI 七层模型与 TCP/IP 四层模型，深入剖析以太网帧、IP 数据报、TCP 报文段及 HTTP 报文的首部结构。
- 理解数据在网络协议栈中的封装（Encapsulation）与解封装过程。

3.对比有线与无线网络中 MAC 帧的差异

- 分析标准以太网（Ethernet II）MAC 帧的帧头结构（目的地址、源地址、类型）。

4.分析 IP、TCP、HTTP 协议的工作机制

- IP 协议：分析 IP 首部的关键字段（如 TTL、Protocol、源/目的 IP），理解网络层寻址机制。
- TCP 协议：通过序列号（Seq）和确认号（Ack）的变化，验证 TCP 三次握手建立连接及四次挥手断开连接的完整状态机流转。
- HTTP 协议：分析 HTTP 请求方法（GET）、响应状态码（200 OK）、头部字段（User-Agent, Host, Content-Type）及明文传输特性。

二、实验环境

1.硬件环境

- 计算机终端：个人笔记本电脑
- 网络适配器：IEEE 802.11 无线网卡（WLAN）

2.软件环境

- 操作系统：Windows 11 24H2
- 抓包分析工具：Wireshark Version 4.6.2
- 底层驱动：Npcap（用于支持数据包捕获及 Monitor Mode 选项支持）
- 网络应用：Google Chrome 浏览器；命令提示符 CMD

3.实验测试对象

- HTTP 协议测试站点：<http://example.com>
- 连通性测试：使用 ping 命令访问公共 DNS 或网关地址，用于捕获 ICMP 及 IP 数据包

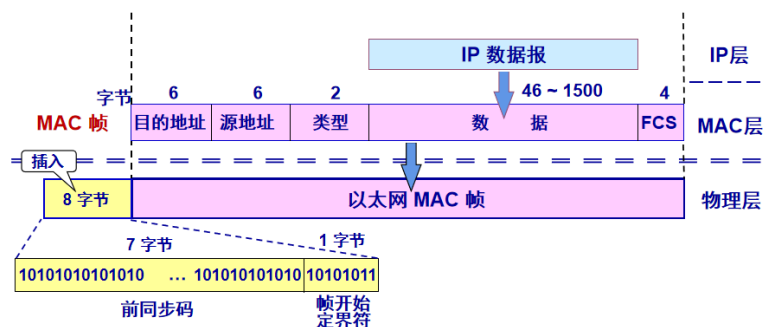
三、捕获并分析以太网 MAC 帧

3.1 以太网 MAC 帧介绍

以太网（Ethernet）是目前应用最广泛的局域网技术，其链路层传输的数据单元被称为以太网帧。目前最通用的以太网帧格式是 Ethernet II（也称为 DIX V2 格式）。

一个标准的 Ethernet II 帧结构包含以下关键字段：

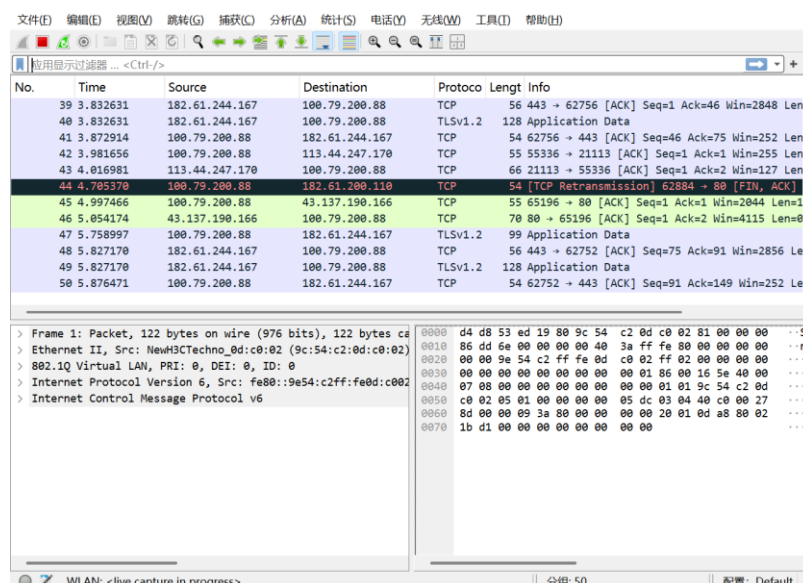
- **前同步码与帧开始定界符**：物理层用于同步信号，通常不被抓包软件捕获。
- **目的地址**：6 字节，标识接收方的物理地址（单播、多播或广播）。
- **源地址**：6 字节，标识发送方的物理地址。
- **类型**：2 字节，用于标识数据字段中封装的上层协议（例如 0x0800 代表 IPv4，0x0806 代表 ARP）。
- **数据**：46~1500 字节，包含网络层的数据报（不足 46 字节会自动填充 Padding）。
- **帧校验序列（FCS）**：4 字节，用于检测传输错误。大多数网卡在硬件层面完成校验并剥离该字段，因此 Wireshark 通常无法显示 FCS。



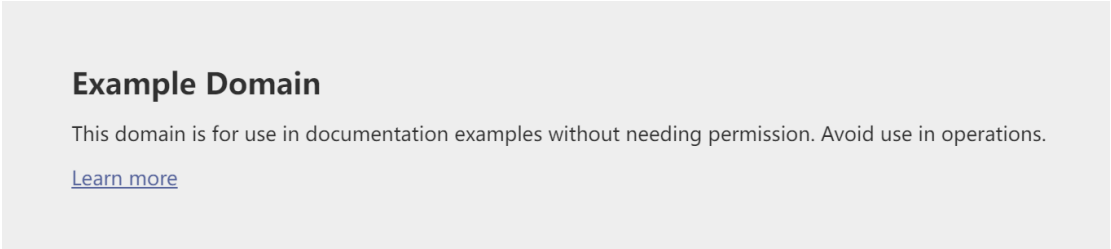
3.2 实验步骤

（1）启动 Wireshark 软件，在主界面选择“WLAN”无线网络接口，双击开始捕获。

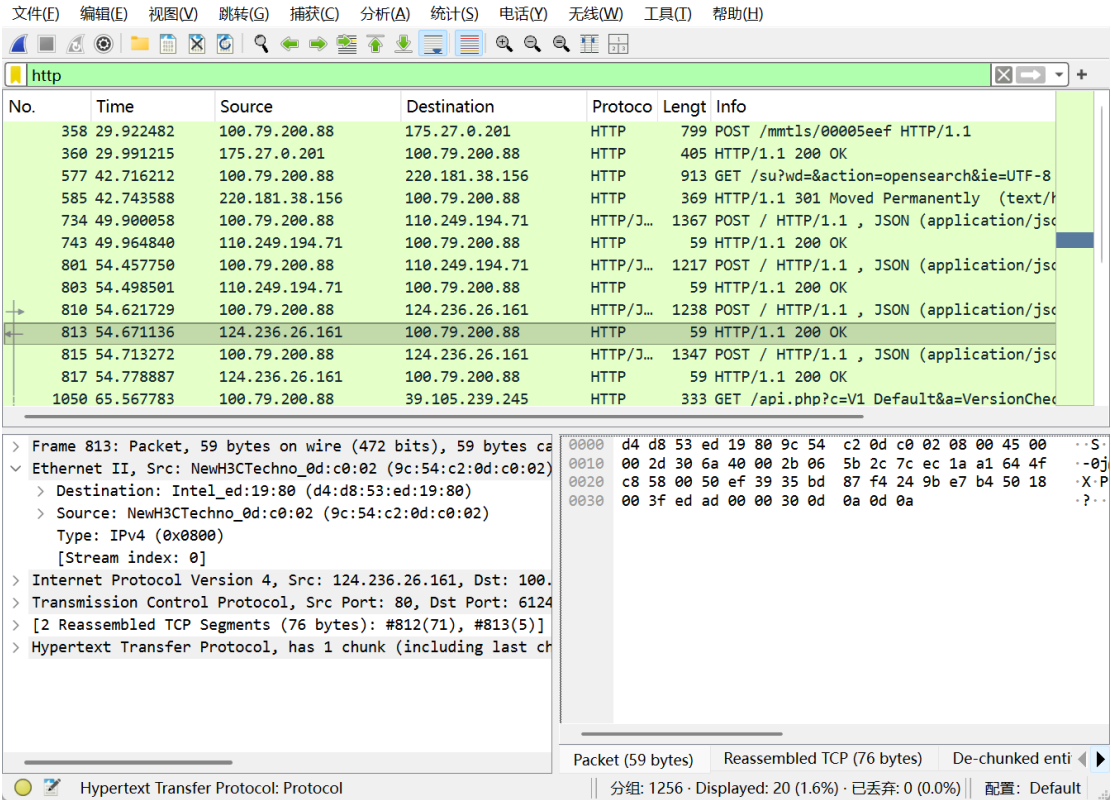
由于 Windows 操作系统的 NDIS 驱动机制，Wireshark 捕获到的 802.11 无线帧通常会被模拟转换为 Ethernet II（以太网）帧格式提交给上层。因此，我们可以直接利用 WLAN 接口来分析以太网帧结构。



(2) 打开浏览器，访问 <http://example.com> 以产生正常的网络通信流量。



(3) 待网页加载完成后停止捕获。在上方的显示过滤器栏中输入 **http** 筛选数据包。在列表选中任意一个数据包，展开 “Ethernet II” 节点，对以太网帧首部字段进行观察与分析。



3.3 帧格式分析

如上图所示，我们分析捕获到编号为 813 的数据帧。该帧为标准的 Ethernet II 格式。可以看到，该帧的首部包含了目的地址 (Destination MAC)、源地址 (Source MAC) 和类型 (Type)。

目的地址的字段值为 d4:d8:53:ed:19:80。这是接收端的物理地址。显示为 Intel 设备，即本实验所使用的笔记本电脑无线网卡。这表明该数据帧是由网关发送给本机的。

源地址的字段值为 9c:54:c2:0d:c0:02。这是发送端的物理地址。

类型字段值 IPv4 (0x0800)。它标识了帧数据字段 (Payload) 中封装的上层协议类型。此处表明以太网帧内部承载的是 IPv4 协议数据包。接收端网卡在处理该帧时，会依据此字段将数据载荷上交给网络层的 IP 协议栈进行处理。

另外，注意到标准以太网帧结构中定义的前同步码、帧开始定界符以及尾部的帧校验序列 (FCS) 均未出现。这是因为网卡在完成同步、校验之后会自动剥离这些字段。

四、 捕获并分析无线局域网 MAC 帧

4.1 无线局域网 MAC 帧介绍

无线局域网最通用的标准是 IEEE 802.11 系列协议，本次实验我们分析的也是 IEEE 802.11 MAC 帧。与以太网只有一种帧格式不同，802.11 MAC 帧分为三大类以管理无线链路：

- **管理帧 (Management Frames)**用于建立和维护无线连接（如 Beacon 信标帧、Probe 探测帧、Association 关联帧）。
- **控制帧 (Control Frames)**用于协助数据传输和信道竞争（如 ACK 确认帧、RTS 请求发送、CTS 清除发送）。
- **数据帧 (Data Frames)**用于正式传输上层（如 IP 层）的数据报文。

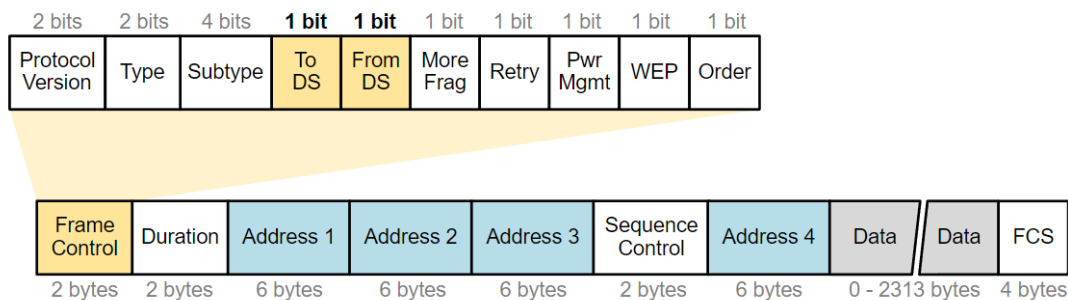


Figure 5. 802.11 Frame Format.

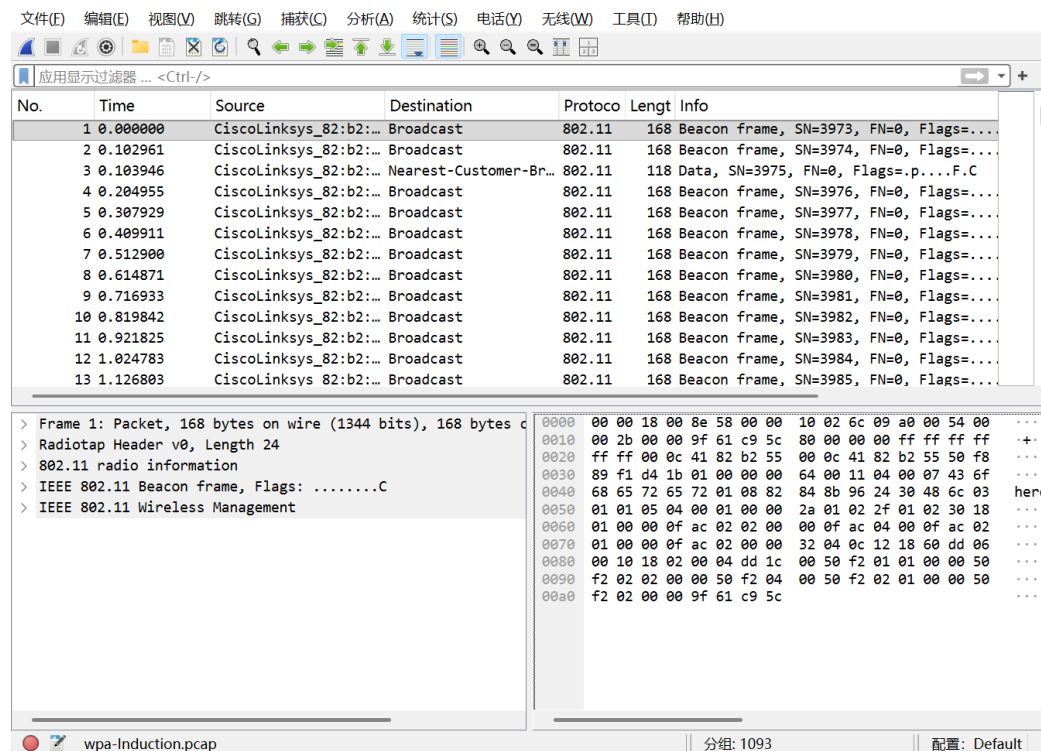
一个通用的 802.11 MAC 帧首部最大可达 30 字节（以太网仅 14 字节），主要包含以下关键字段：

- **帧控制 (Frame Control, 2 字节)**：包含协议版本、帧类型 (Type)、子类型 (Subtype)、去往/来自 DS（分布式系统）等控制位。
- **持续时间 (Duration, 2 字节)**：用于虚拟载波监听机制 (NAV)，告知其他站点信道将被占用多久。
- **地址字段 (Address 1~4, 每个 6 字节)**：=802.11 帧最多定义了 4 个 MAC 地址字段，分别用于标识：
 - **DA (Destination Address)**：最终目的地址。
 - **SA (Source Address)**：原始源地址。
 - **RA (Receiver Address)**：下一跳接收端。
 - **TA (Transmitter Address)**：上一跳发送端。
- **序列控制 (Sequence Control, 2 字节)**：用于帧的排序和碎片重组。
- **帧校验序列 (FCS, 4 字节)**：用于检错。

4.2 实验步骤

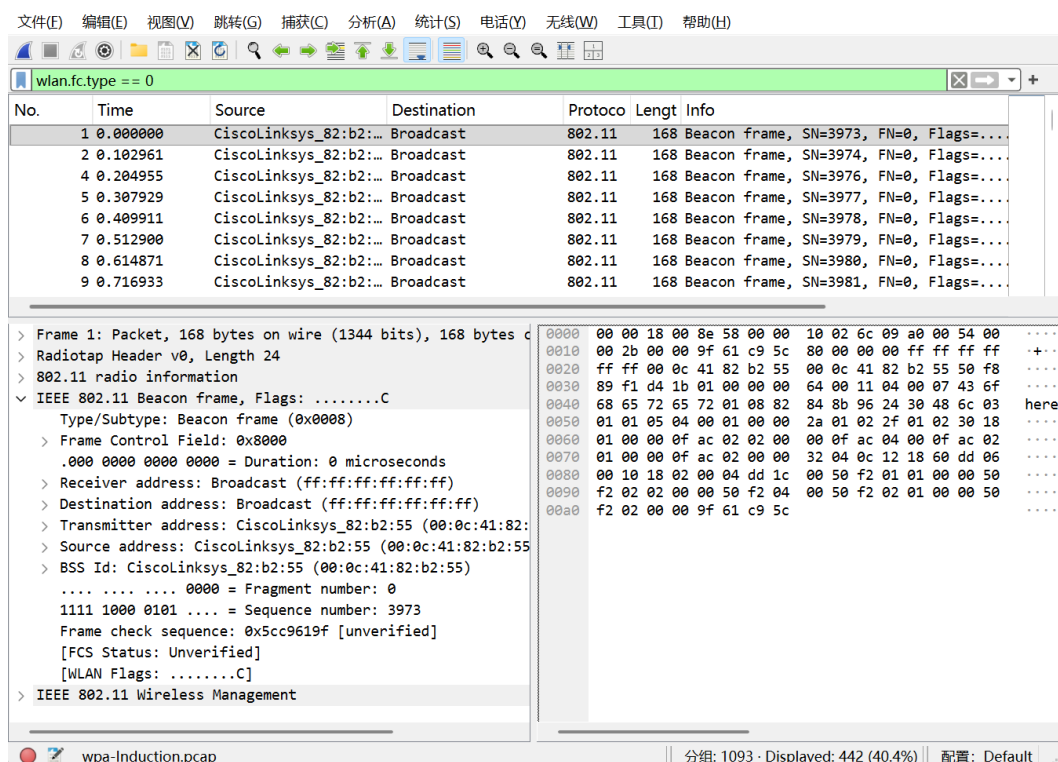
由于 Windows 操作系统下的无线网卡驱动在普通模式下无法向 Wireshark 提供原始的 802.11 帧（会被转换为以太网帧），为了观察标准的无线 MAC 帧结构，本实验采用 Wireshark 官方提供的标准无线抓包示例文件进行分析。

获取标准数据包文件：下载 Wireshark 官方 Wiki 提供的无线局域网示例文件 wpa-Induction.pcap，在 wireshark 中打开



使用过滤器分类查看三种类型的无线局域网 MAC 帧。针对每种筛选出的典型帧，在下方详情窗口展开 IEEE 802.11 头部及 Frame Control 字段，观察其子类型（Subtype）和地址字段的差异

筛选管理帧（wlan.fc.type == 0）



筛选控制帧 (wlan.fc.type == 1)

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(I) 帮助(H)

wlan.fc.type == 1

No.	Time	Source	Destination	Protocol	Length	Info
18	1.608711		CiscoLinksys_82:b2:...	802.11	38	Acknowledgement, Flags=.....C
60	5.183030		CiscoLinksys_82:b2:...	802.11	38	Acknowledgement, Flags=.....C
63	5.203028		CiscoLinksys_82:b2:...	802.11	38	Acknowledgement, Flags=.....C
79	5.644038		Apple_82:36:3a	802.11	38	Acknowledgement, Flags=.....C
81	5.645039		CiscoLinksys_82:b2:...	802.11	38	Acknowledgement, Flags=.....C
83	5.646955		Apple_82:36:3a	802.11	38	Acknowledgement, Flags=.....C
85	5.647962		CiscoLinksys_82:b2:...	802.11	38	Acknowledgement, Flags=.....C
86	5.648961		CiscoLinksys_82:b2:...	802.11	38	Clear-to-send, Flags=.....C

> Frame 18: Packet, 38 bytes on wire (304 bits), 38 bytes captured on interface 0

> Radiotap Header v0, Length 24

> 802.11 radio information

> IEEE 802.11 Acknowledgement, Flags:C

Type/Subtype: Acknowledgement (0x001d)

> Frame Control Field: 0xd400

.000 0000 0000 0000 = Duration: 0 microseconds

> Receiver address: CiscoLinksys_82:b2:55 (00:0c:41:82:b2:55)

Frame check sequence: 0x7c6b3b3 [unverified]

[FCS Status: Unverified]

[WLAN Flags:C]

wpa-Induction.pcap 分组: 1093 · Displayed: 356 (32.6%) 配置: Default

筛选数据帧 (wlan.fc.type == 2)

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(I) 帮助(H)

wlan.fc.type == 2

No.	Time	Source	Destination	Protocol	Length	Info
3	0.103946	CiscoLinksys_82:b2:...	Nearest-Customer-Br...	802.11	118	Data, SN=3975, FN=0, Flags=.p....F.C
26	2.151616	CiscoLinksys_82:b2:...	Nearest-Customer-Br...	802.11	118	Data, SN=4000, FN=0, Flags=.p....F.C
47	4.098227	CiscoLinksys_82:b2:...	Nearest-Customer-Br...	802.11	118	Data, SN=4020, FN=0, Flags=.p....F.C
87	5.649953	CiscoLinksys_82:b2:...	Apple_82:36:3a	EAPOL	181	Key (Message 1 of 4)
89	5.650959	Apple_82:36:3a	CiscoLinksys_82:b2:...	EAPOL	181	Key (Message 2 of 4)
92	5.655957	CiscoLinksys_82:b2:...	Apple_82:36:3a	EAPOL	239	Key (Message 3 of 4)
94	5.655973	Apple_82:36:3a	CiscoLinksys_82:b2:...	EAPOL	159	Key (Message 4 of 4)
99	5.844024	Apple_82:36:3a	Broadcast	802.11	404	Data, SN=27, FN=0, Flags=.p....TC

> 802.11 radio information

> IEEE 802.11 Data, Flags: .p....F.C

Type/Subtype: Data (0x0020)

> Frame Control Field: 0x0842

.000 0000 0000 0000 = Duration: 0 microseconds

> Receiver address: Nearest-Customer-Bridge (01:80:c2:00:00:00)

> Transmitter address: CiscoLinksys_82:b2:55 (00:0c:41:82:b2:55)

> Destination address: Nearest-Customer-Bridge (01:80:c2:00:00:00)

> Source address: CiscoLinksys_82:b2:55 (00:0c:41:82:b2:55)

> BSS Id: CiscoLinksys_82:b2:55 (00:0c:41:82:b2:55)

> STA address: Nearest-Customer-Bridge (01:80:c2:00:00:00)

.... 0000 = Fragment number: 0

1111 1000 0111 = Sequence number: 3975

Frame check sequence: 0xe5e03e71 [unverified]

[FCS Status: Unverified]

[WLAN Flags: .p....F.C]

> TKIP parameters

> Data (58 bytes)

wpa-Induction.pcap 分组: 1093 · Displayed: 285 (26.1%) 配置: Default

4.3 帧格式分析

4.3.1 管理帧

分析帧 No.1，这是一个典型的 802.11 信标帧。

详情栏显示 Type/Subtype: Beacon frame (0x0008)，这表明 Type 字段为 0 (Management)，Subtype 字段为 8 (Beacon)。

Destination address (DA): Broadcast (ff:ff:ff:ff:ff:ff)，表明该帧是向所有设备广播的。

Source address (SA): CiscoLinksys_82:b2:55(00:0c:41:82:b2:55)，表明发送者是 Cisco 无线接入点 (AP)。

BSS Id: CiscoLinksys_82:b2:55(00:0c:41:82:b2:55)，表明该无线网络的基本服务集 ID 与 AP 的 MAC 地址一致。

4.3.2 控制帧

分析帧 No.18，这是一个 ACK 确认帧。

详情栏显示 Type/Subtype: Acknowledgement (0x001d)，这表明 Type 字段为 1 (Control)，Subtype 字段为 13，代表 ACK。

控制帧头部非常短，从图中可以看到它只有一个地址字段：Receiver address (RA): CiscoLinksys_82:b2:55(00:0c:41:82:b2:55)。这表明该确认帧是发给刚才那个 Cisco AP 的，用于确认已成功收到其发送的数据。

4.3.3 数据帧

分析帧 No.3，这是一个 802.11 数据帧。

详情栏显示 Type/Subtype: Data (0x0020)，这表明 Type 字段为 2 (Data)，Subtype 字段为 0 (Data)。

从图中可以发现，Frame Control Field: 0x0842。From DS: 1 (AP 发送)，To DS: 0 (非发往 AP)，所以这是一个从无线接入点 (AP) 发往接收端 (STA) 的下行数据帧。

Receiver address (RA/DA): Nearest-Customer-Bridge，这是一个组播/桥接地址，表示接收端。

Transmitter address (TA): CiscoLinksys_82:b2:55，表示发送该信号的 AP。

Source address (SA): CiscoLinksys_82:b2:55，表示原始数据源也是该 AP。

五、IP 协议分析

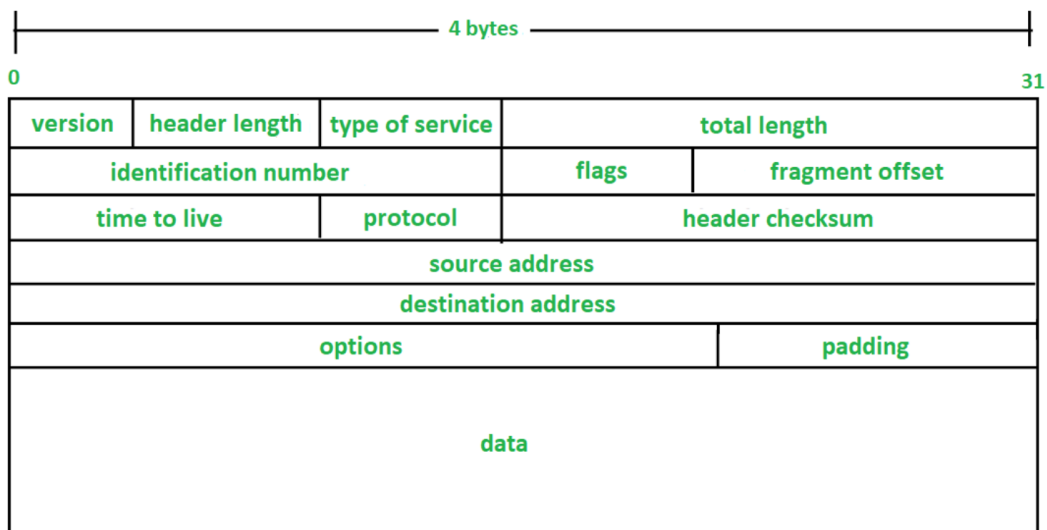
5.1 IP 协议介绍

网际协议 (Internet Protocol, IP) 是 TCP/IP 协议栈的核心，工作在 OSI 模型的第三层 (网络层)，其任务是根据源主机和目的主机的地址传送数据。目前广泛使用的是 IPv4 版本，本次实验分析的也是这个版本。IP 协议提供的是不可靠、无连接的数据报投递服务。

一个标准的 IPv4 数据报首部长度通常为 20 字节，包含以下关键字段：

- **版本 (Version):** 4 比特，指明协议版本 (IPv4 为 4)。
- **首部长度 (Header Length):** 4 比特，指明首部的长度 (单位为 4 字节)，最小值为 5 (即 20 字节)。
- **区分服务 (Differentiated Services):** 用于 QoS 服务质量控制。
- **总长度 (Total Length):** 16 比特，指明整个 IP 数据报的长度。

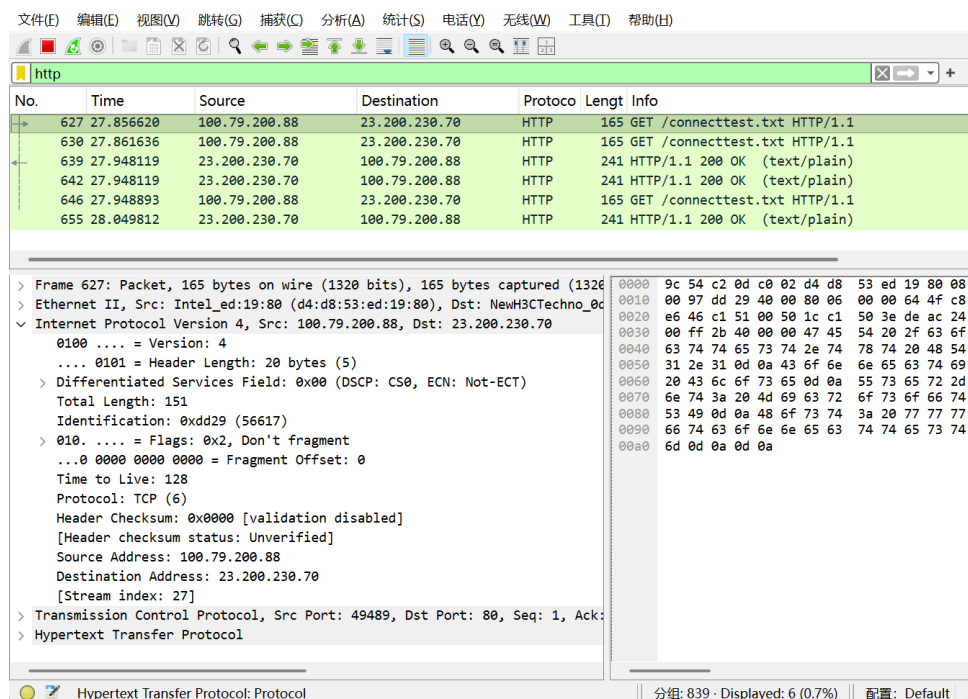
- **标识、标志、片偏移：**用于 IP 数据报的分片与重组。
- **生存时间 (TTL)：**8 比特，防止数据报在网络中无限循环，每经过一个路由器减 1。
- **协议 (Protocol)：**8 比特，指明上层协议类型（如 ICMP=1, TCP=6, UDP=17）。
- **首部校验和 (Header Checksum)：**仅检验首部的正确性。
- **源/目的 IP 地址：**各 32 比特，标识发送端和接收端。



IPv4 Header format

5.2 实验步骤

- (1) 启动 Wireshark，选择 “WLAN” 接口开始捕获。
- (2) 打开浏览器，访问 <http://example.com>，产生应用层流量。待网页加载完毕后，停止 Wireshark 捕获。
- (3) 在显示过滤器中输入 `http`，筛选出 HTTP 请求或响应报文。选中其中任意一个数据包，在详情窗口找到并展开 Internet Protocol Version 4 (IP 协议首部) 进行分析。



5.3 IP 报文格式分析

如图所示，本次实验选取了编号为 627 的 HTTP 请求数据包进行 IP 首部详细分析。

1. 版本 (Version) 字段值 4。二进制为 0100，表示该数据报使用的是 IPv4 协议。

2. 首部长度 (Header Length) 字段值 20 bytes (5)。字段值为 5，单位为 4 字节。 $5 \times 4 = 20$ 字节，说明该 IP 首部不包含可选字段 (Options)，是最常用的标准长度。

3. DS 部分 (0x00) 表示差分服务代码点 (DSCP) 为 CS0，无显示拥塞通告 (ECN)；

4. 总长度 (Total Length) 字段值 151。表示整个 IP 数据报 (首部 + 数据) 的总长度为 151 字节。计算载荷： $151 - 20(\text{IP 头}) = 131$ 字节。这 131 字节就是封装在 IP 包内部的 TCP 报文段 (包含 TCP 头和 HTTP 请求数据)。

5. 标识 (Identification) 字段值 0xdd29 (56617)。这是一个唯一的计数器值，用于在数据报分片重组时标识属于同一数据流的分片。

6. 标志 (Flags) 字段值：0x2, Don't fragment。DF (Don't Fragment) 位置 1 表示强制要求路由器不要对该包进行分片。MF (More Fragments) 位置 0 表示没有后续分片。

7. 生存时间 (Time to Live, TTL) 字段值 128。表示该数据报在网络中最多能经过 128 个路由器。这是 Windows 系统发送数据包的默认 TTL 值。

8. 协议 (Protocol) 字段值 TCP (6)。指出 IP 数据报的数据区封装的是 TCP 协议。

9. 源与目的 IP 地址

- 源 IP (Source): 100.79.200.88 (本机地址)。
- 目的 IP (Destination): 23.200.230.70 (目标服务器地址)。

六、TCP 协议分析

6.1 TCP 协议介绍

传输控制协议 (TCP) 是一种面向连接的、可靠的、基于字节流的传输层通信协议。它通过“三次握手”建立连接，通过“四次挥手”释放连接，并利用序列号、确认应答、重传控制、流量控制 (滑动窗口) 和拥塞控制等机制来保证数据传输的可靠性。在数据传输开始前，通信双方必须先建立连接，这通过著名的“三次握手”过程实现：

- 第一次握手 (SYN): 客户端发送 SYN 报文，请求建立连接。
- 第二次握手 (SYN, ACK): 服务器回复 SYN+ACK 报文，表示同意连接并确认收到请求。
- 第三次握手 (ACK): 客户端回复 ACK 报文，连接建立成功。

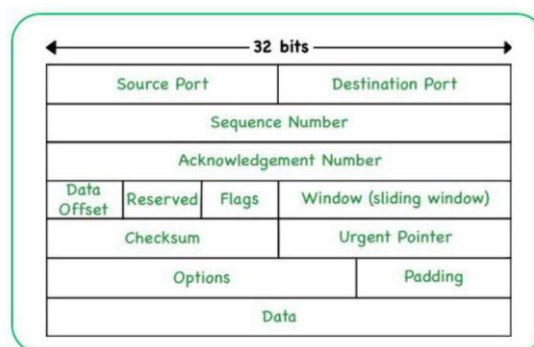


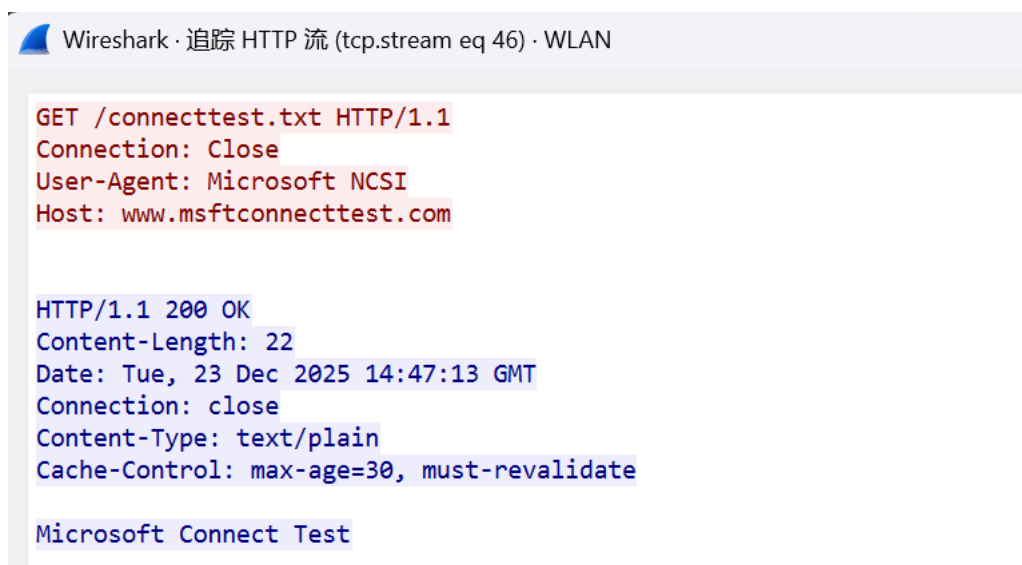
Diagram Showing the TCP packet Format

如上图所示，TCP 首部通常为 20 字节（不含选项），各字段含义如下：

- **源端口 (Source Port) 和 目的端口 (Destination Port):** 各占 16 位。用于标识发送方和接收方的应用进程（例如 HTTP 默认使用 80 端口，HTTPS 使用 443 端口）。
- **序列号 (Sequence Number):** 占 32 位。用于标识本报文段所发送数据的第一个字节的序号，保证数据的有序传输。
- **确认号 (Acknowledgement Number):** 占 32 位。期望收到对方下一个报文段的第一个数据字节的序号。若确认号为 N，则表明到序号 N-1 为止的所有数据都已正确收到。
- **数据偏移 (Data Offset):** 占 4 位。指出 TCP 首部的长度（以 4 字节为单位）。最小值为 5（即 20 字节）。
- **保留 (Reserved):** 占 6 位，保留为今后使用，目前置为 0。
- **标志位 (Flags):** 用于控制协议的状态机。常见的标志位包括：
 - **SYN:** 同步位，用于建立连接。
 - **ACK:** 确认位，当该位为 1 时确认号字段才有效。
 - **FIN:** 终止位，用于释放连接。
 - **RST:** 复位位，用于重置异常连接。
- **窗口 (Window):** 占 16 位。用于流量控制，指明发送本报文段的一方的接收窗口大小，即告诉对方现在还能接收多少数据。
- **校验和 (Checksum):** 占 16 位。用于检验首部和数据部分的正确性。
- **紧急指针 (Urgent Pointer):** 占 16 位。当 URG 标志置 1 时有效，指出紧急数据的末尾在报文段中的位置。
- **选项 (Options) 和 填充 (Padding):** 长度可变。最常见的选项是最大报文段长度 (MSS)。

6.2 实验步骤

之前分析 IP 协议时，我们抓 627 号帧进行分析。接着我们在 Wireshark 中对刚才抓取的 HTTP 数据流进行追踪（Follow TCP Stream）



```
Wireshark · 追踪 HTTP 流 (tcp.stream eq 46) · WLAN

GET /connecttest.txt HTTP/1.1
Connection: Close
User-Agent: Microsoft NCSI
Host: www.msftconnecttest.com

HTTP/1.1 200 OK
Content-Length: 22
Date: Tue, 23 Dec 2025 14:47:13 GMT
Connection: close
Content-Type: text/plain
Cache-Control: max-age=30, must-revalidate

Microsoft Connect Test
```

观察数据包列表窗格，定位到 TCP 连接建立的前三个数据包（No. 602, 603, 604），这就是三

次握手的三个包，分别是 SYN、SYN ACK 和 ACK。

TCP 连接的最后四个包，就是四次挥手的四个包，分别是 FIN、ACK、FIN、ACK。

No.	Time	Source	Destination	Protocol	Length	Info
602	27.791042	100.79.200.88	23.200.230.70	TCP	66	49489 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=14
625	27.855979	23.200.230.70	100.79.200.88	TCP	66	80 → 49489 [SYN, ACK] Seq=0 Ack=1 Win=64240 L
626	27.856113	100.79.200.88	23.200.230.70	TCP	54	49489 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
627	27.856620	100.79.200.88	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
638	27.948119	23.200.230.70	100.79.200.88	TCP	56	80 → 49489 [ACK] Seq=1 Ack=112 Win=64256 Len=0
639	27.948119	23.200.230.70	100.79.200.88	HTTP	241	HTTP/1.1 200 OK (text/plain)
640	27.948119	23.200.230.70	100.79.200.88	TCP	56	80 → 49489 [FIN, ACK] Seq=188 Ack=112 Win=642
645	27.948908	100.79.200.88	23.200.230.70	TCP	54	49489 → 80 [ACK] Seq=112 Ack=189 Win=65280 Le
648	27.949191	100.79.200.88	23.200.230.70	TCP	54	49489 → 80 [FIN, ACK] Seq=112 Ack=189 Win=652
653	28.049812	23.200.230.70	100.79.200.88	TCP	56	80 → 49489 [ACK] Seq=189 Ack=113 Win=64256 Le

6.3 三次握手过程

第一次握手：客户端发送连接请求 (SYN)

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(I) 帮助(H)

tcp.stream eq 46

No.	Time	Source	Destination	Protocol	Length	Info
602	27.791042	100.79.200.88	23.200.230.70	TCP	66	49489 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=
625	27.855979	23.200.230.70	100.79.200.88	TCP	66	80 → 49489 [SYN, ACK] Seq=0 Ack=1 Win=64240
626	27.856113	100.79.200.88	23.200.230.70	TCP	54	49489 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=
627	27.856620	100.79.200.88	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
638	27.948119	23.200.230.70	100.79.200.88	TCP	56	80 → 49489 [ACK] Seq=1 Ack=112 Win=64256 Le
639	27.948119	23.200.230.70	100.79.200.88	HTTP	241	HTTP/1.1 200 OK (text/plain)

> Frame 602: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bit)
> Ethernet II, Src: Intel_ed:19:80 (d4:d8:53:ed:19:80), Dst: NewH3CTechno_0c
> Internet Protocol Version 4, Src: 100.79.200.88, Dst: 23.200.230.70
✖ Transmission Control Protocol, Src Port: 49489, Dst Port: 80, Seq: 0, Len: 0

Source Port: 49489
Destination Port: 80
[Stream index: 46]
[Stream Packet Number: 1]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 482431037
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1000 = Header Length: 32 bytes (8)
Flags: 0x002 (SYN)
Window: 65535
[Calculated window size: 65535]
Checksum: 0x2add [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window s
[Timestamps]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]

0000 9c 54 c2 0d c0 02 d4 d8 53 ed 19 80 08
0010 00 34 dd 25 40 00 80 06 00 00 64 4f c8
0020 e6 46 c1 51 00 50 1c c1 50 3d 00 00 00
0030 ff ff 2a dd 00 00 02 04 05 b4 01 03 03
0040 04 02

wireshark_WLANFT9QH3.pcapng 分组: 59590 · Displayed: 10 (0.0%) 配置: Default

- 数据包编号: No. 602
- 方向: 客户端→服务器
- 标志位 (Flags): 0x002 (SYN)。SYN=1, ACK=0 表示这是一个连接请求报文。
- 序列号 (Seq): 0 (relative)。客户端随机生成一个初始序列号 (x)，Wireshark 显示相对值为 0。
- 窗口大小 (Window): 65535

分析: 客户端通告其初始接收窗口为 65535 字节。这意味着客户端告诉服务器: 我的

接收缓冲区目前有 65KB 的空闲空间。

此时，客户端向服务器的 80 端口发起连接，进入 **SYN_SENT** 状态，等待服务器确认。

第二次握手：服务器确认并请求连接 (SYN+ACK)

The image shows a Wireshark packet capture of a TCP connection. The packet list at the top shows several packets, with packet 625 highlighted. The packet details pane on the left shows the structure of packet 625, which is a TCP segment. The packet bytes pane on the right shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
602	27.791042	100.79.200.88	23.200.230.70	TCP	66	49489 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=
625	27.855979	23.200.230.70	100.79.200.88	TCP	66	80 → 49489 [SYN, ACK] Seq=0 Ack=1 Win=64240
626	27.856113	100.79.200.88	23.200.230.70	TCP	54	49489 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=
627	27.856620	100.79.200.88	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
638	27.948119	23.200.230.70	100.79.200.88	TCP	56	80 → 49489 [ACK] Seq=1 Ack=112 Win=64256 Le
639	27.948119	23.200.230.70	100.79.200.88	HTTP	241	HTTP/1.1 200 OK (text/plain)

Packet 625 details:

- Frame 625: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 b)
- Ethernet II, Src: NewH3CTechno_0d:c0:02 (9c:54:c2:0d:c0:02), Dst: Intel
- Internet Protocol Version 4, Src: 23.200.230.70, Dst: 100.79.200.88
- Transmission Control Protocol, Src Port: 80, Dst Port: 49489, Seq: 0, A
- Source Port: 80
- Destination Port: 49489
- [Stream index: 46]
- [Stream Packet Number: 2]
- [Conversation completeness: Complete, WITH_DATA (31)]
- [TCP Segment Len: 0]
- Sequence Number: 0 (relative sequence number)
- Sequence Number (raw): 3735823526
- [Next Sequence Number: 1 (relative sequence number)]
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment number (raw): 482431038
- 1000 = Header Length: 32 bytes (8)
- Flags: 0x012 (SYN, ACK)
- Window: 64240
- [Calculated window size: 64240]
- Checksum: 0x17b0 [unverified]
- [Checksum Status: Unverified]
- Urgent Pointer: 0
- Options: (12 bytes), Maximum segment size, No-Operation (NOP), No-Op
- [Timestamps]
- [SEQ/ACK analysis]
- [Client Contiguous Streams: 1]
- [Server Contiguous Streams: 1]

Packet bytes:

```
0000 d4 d8 53 ed 19 80 9c 54 c2 0d c0 02 08
0010 00 34 00 00 40 00 2e 06 22 0e 17 c8 e6
0020 c8 58 00 50 c1 51 de ac 24 a6 1c c1 50
0030 fa f0 17 b0 00 00 02 04 05 6a 01 01 04
0040 03 07
```

- **数据包编号 (No.):** 625
 - **方向:** 服务器 → 客户端
 - **标志位 (Flags):** 0x012 (SYN, ACK)
 - **SYN=1:** 服务器也发起连接请求。
 - **ACK=1:** 服务器确认收到了客户端的请求。
 - **确认号 (Ack):** 1。计算公式为客户端的 $Seq + 1$ (即 $0 + 1 = 1$)，表明服务器期望收到客户端的下一个字节序号。
 - **序列号 (Seq):** 0 (relative)。
 - **窗口大小 (Window):** 65535。
 - **分析:** 客户端通告其初始接收窗口为 65535 字节。这意味着客户端告诉服务器：我的接收缓冲区目前有 65KB 的空闲空间。
 - 服务器生成自己的初始序列号 (y)，相对值显示为 0。
- 此时，服务器收到 SYN 包后，回复 SYN+ACK 包，进入 **SYN_RCVD** 状态。

第三次握手：客户端确认 (ACK)

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(I) 帮助(H)

tcp.stream eq 46

No.	Time	Source	Destination	Protocol	Length	Info
602	27.791042	100.79.200.88	23.200.230.70	TCP	66	49489 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=
625	27.855979	23.200.230.70	100.79.200.88	TCP	66	80 → 49489 [SYN, ACK] Seq=0 Ack=1 Win=64240
626	27.856113	100.79.200.88	23.200.230.70	TCP	54	49489 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=
627	27.856620	100.79.200.88	23.200.230.70	HTTP	165	GET /connecttest.txt HTTP/1.1
638	27.948119	23.200.230.70	100.79.200.88	TCP	56	80 → 49489 [ACK] Seq=1 Ack=112 Win=64256 Le
639	27.948119	23.200.230.70	100.79.200.88	HTTP	241	HTTP/1.1 200 OK (text/plain)

> Frame 626: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 b)
> Ethernet II, Src: Intel_ed:19:80 (d4:d8:53:ed:19:80), Dst: NewH3CTechno.
> Internet Protocol Version 4, Src: 100.79.200.88, Dst: 23.200.230.70
✓ Transmission Control Protocol, Src Port: 49489, Dst Port: 80, Seq: 1, A
 Source Port: 49489
 Destination Port: 80
 [Stream index: 46]
 [Stream Packet Number: 3]
 > [Conversation completeness: Complete, WITH_DATA (31)]
 [TCP Segment Len: 0]
 Sequence Number: 1 (relative sequence number)
 Sequence Number (raw): 482431038
 [Next Sequence Number: 1 (relative sequence number)]
 Acknowledgment Number: 1 (relative ack number)
 Acknowledgment number (raw): 3735823527
 0101 = Header Length: 20 bytes (5)
 > Flags: 0x010 (ACK)
 Window: 255
 [Calculated window size: 65280]
 [Window size scaling factor: 256]
 Checksum: 0x2ad1 [unverified]
 [Checksum Status: Unverified]
 Urgent Pointer: 0
 > [Timestamps]
 > [SEQ/ACK analysis]
 [Client Contiguous Streams: 1]
 [Server Contiguous Streams: 1]

0000 9c 54 c2 0d c0 02 d4 d8 53 ed 19 80 08
0010 00 28 dd 28 40 00 80 06 00 00 64 4f c8
0020 e6 46 c1 51 00 50 1c c1 50 3e de ac 24
0030 00 ff 2a d1 00 00

wireshark_WLANFT9QH3.pcapng 分组: 61402 · Displayed: 10 (0.0%) 配置: Default

- 数据包编号：No. 626
- 方向：客户端 → 服务器
- 标志位 (Flags)：0x010 (ACK)。SYN=0, ACK=1。连接建立完成，不再需要 SYN 标志。
- 确认号 (Ack)：1。
- 计算公式为服务器的 Seq + 1（即 0 + 1 = 1），表明客户端已收到服务器的 SYN。
- 序列号 (Seq)：1。客户端消耗了一个序列号用于握手，现在的序列号变为 1。
- 窗口大小 (Window)：65280。客户端根据系统资源情况，微调并更新了自己的接收窗口大小。

此时，客户端收到 SYN+ACK 包后发送确认，进入 ESTABLISHED 状态。此时服务器收到该包后也进入 ESTABLISHED 状态，TCP 连接正式建立，双方可以开始传输 HTTP 数据（如随后的 Packet 627）。

6.4 TCP 连接断开过程（四次挥手）分析

我们观察到，在 HTTP 响应传输结束后，连接进入断开阶段。本次抓包捕获了完整的断开过程（Packet 640 - 653）

文件(F) 编辑(E) 视图(V) 跳转(G) 捕获(C) 分析(A) 统计(S) 电话(Y) 无线(W) 工具(I) 帮助(H)

tcp.stream eq 46

No.	Time	Source	Destination	Protocol	Length	Info
639	27.948119	23.200.230.70	100.79.200.88	HTTP	241	HTTP/1.1 200 OK (text/plain)
640	27.948119	23.200.230.70	100.79.200.88	TCP	56	80 → 49489 [FIN, ACK] Seq=188 Ack=112 Win=6
645	27.948908	100.79.200.88	23.200.230.70	TCP	54	49489 → 80 [ACK] Seq=112 Ack=189 Win=65280
648	27.949191	100.79.200.88	23.200.230.70	TCP	54	49489 → 80 [FIN, ACK] Seq=112 Ack=189 Win=6
653	28.049812	23.200.230.70	100.79.200.88	TCP	56	80 → 49489 [ACK] Seq=189 Ack=113 Win=64256

> Frame 640: Packet, 56 bytes on wire (448 bits), 56 bytes captured (448 bit)

> Ethernet II, Src: NewH3CTechno_0d:c0:02 (9c:54:c2:0d:c0:02), Dst: Intel_ec

> Internet Protocol Version 4, Src: 23.200.230.70, Dst: 100.79.200.88

> Transmission Control Protocol, Src Port: 80, Dst Port: 49489, Seq: 188, Ac

Source Port: 80

Destination Port: 49489

[Stream index: 46]

[Stream Packet Number: 7]

> [Conversation completeness: Complete, WITH_DATA (31)]

[TCP Segment Len: 0]

Sequence Number: 188 (relative sequence number)

Sequence Number (raw): 3735823714

[Next Sequence Number: 189 (relative sequence number)]

Acknowledgment Number: 112 (relative ack number)

Acknowledgment number (raw): 482431149

0101 = Header Length: 20 bytes (5)

> Flags: 0x011 (FIN, ACK)

Window: 502

[Calculated window size: 64256]

[Window size scaling factor: 128]

Checksum: 0x5008 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

> [Timestamps]

[Client Contiguous Streams: 1]

[Server Contiguous Streams: 1]

0000 d4 d8 53 ed 19 80 9c 54 c2 0d c0 02 08

0010 00 28 e6 af 40 00 2e 06 3b 6a 17 c8 e6

0020 c8 58 00 50 c1 51 de ac 25 62 1c c1 50

0030 01 f6 50 08 00 00 00 00

wireshark WLANFT9QH3.pcapng 分组: 174962 · Displayed: 10 (0.0%) 配置: Default

1. 第一次挥手 (FIN): 服务器发起关闭

- 数据包: No. 640
- 方向: 服务器 (23.200.230.70) → 客户端。
- 标志位: Flags: 0x011 (FIN, ACK)。
 - **FIN=1**: 表示服务器的数据已经发送完毕, 请求释放连接。
 - **Seq=188**: 服务器当前的序列号。
- 分析: 服务器进入 **FIN_WAIT_1** 状态, 等待客户端确认。

2. 第二次挥手 (ACK): 客户端确认

- 数据包: No. 645。
- 方向: 客户端 → 服务器。
- 内容: 发送 ACK 包, 确认号 Ack=189 (188+1)。
- 分析: 客户端确认收到了服务器的 FIN。此时 TCP 连接处于半关闭状态 (服务器不再发数据, 但仍可接收)。客户端进入 **CLOSE_WAIT** 状态。

3. 第三次挥手 (FIN): 客户端发起关闭

- 数据包: No. 648。
- 方向: 客户端 → 服务器。
- 内容: 发送 FIN, ACK 包。
- 分析: 客户端应用层 (NCSI) 处理完数据后, 也决定关闭连接。客户端进入 **LAST_ACK** 状态。

4. 第四次挥手 (ACK): 服务器确认

- **数据包:** No. 653。
- **方向:** 服务器 → 客户端。
- **内容:** 发送 ACK 包。
- **分析:** 服务器收到客户端的 FIN 后回复确认。连接彻底关闭。

七、HTTP 协议分析

7.1 HTTP 协议介绍

超文本传输协议 (HTTP, HyperText Transfer Protocol) 是互联网上应用最为广泛的一种网络协议, 工作在 OSI 模型的应用层。它定义了客户端 (如浏览器) 与服务器之间交换报文的格式和方式。HTTP 使用 TCP 作为传输层协议 (通常使用 80 端口), 保证了数据的可靠传输。此外, HTTP 是一种无状态 (Stateless) 协议, 服务器不保存关于客户的任何状态信息。

HTTP 报文分为两类: 请求报文 (Request) 和 响应报文 (Response)。虽然两者作用不同, 但其结构逻辑高度一致, 均由三部分组成: 开始行、首部行和实体主体。

HTTP 请求报文通用格式:

- **请求行 (Request Line):** 包含三个字段:
 - **方法 (Method):** 如 GET (获取资源)、POST (提交数据) 等。
 - **URL:** 请求资源的路径。
 - **版本 (Version):** 使用的 HTTP 协议版本 (如 HTTP/1.1)。
- **首部行 (Header Lines):** 包含若干个属性, 用于说明主机域名 (Host)、用户代理 (User-Agent)、连接状态 (Connection) 等。
- **空行 (Blank Line):** 一个回车换行符 (CRLF), 用于分隔首部和实体主体。
- **实体主体 (Entity Body):** 在 GET 请求中通常为空白。

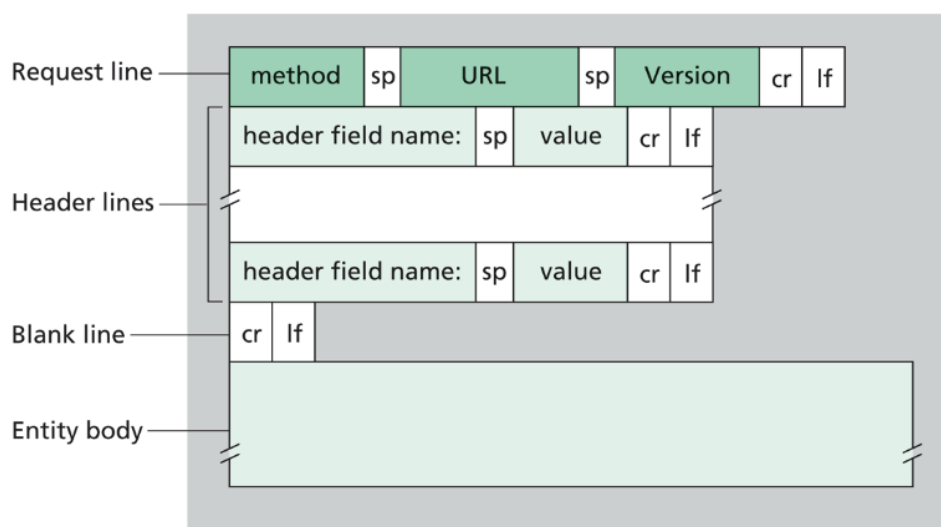


Figure 2.8 ♦ General format of a request message

HTTP 响应报文通用格式:

- 状态行 (Status Line): 包含三个字段:
 - 版本 (Version): 协议版本。
 - 状态码 (Status Code): 如 200 (成功)、404 (未找到)、301 (重定向)。
 - 短语 (Phrase): 状态码的简单文本描述 (如 OK, Not Found)。
- 首部行 (Header Lines): 包含服务器信息、内容类型 (Content-Type)、内容长度 (Content-Length) 等。
- 空行: 分隔符。
- 实体主体 (Entity Body): 也就是服务器返回的实际数据 (如 HTML 网页代码、图片数据或本文实验中的纯文本)。

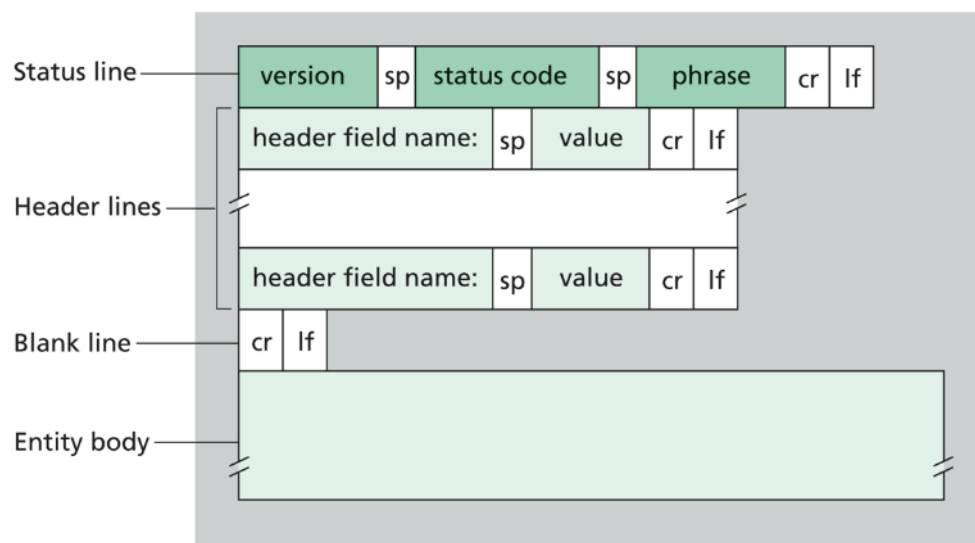


Figure 2.9 ♦ General format of a response message

7.2 实验步骤

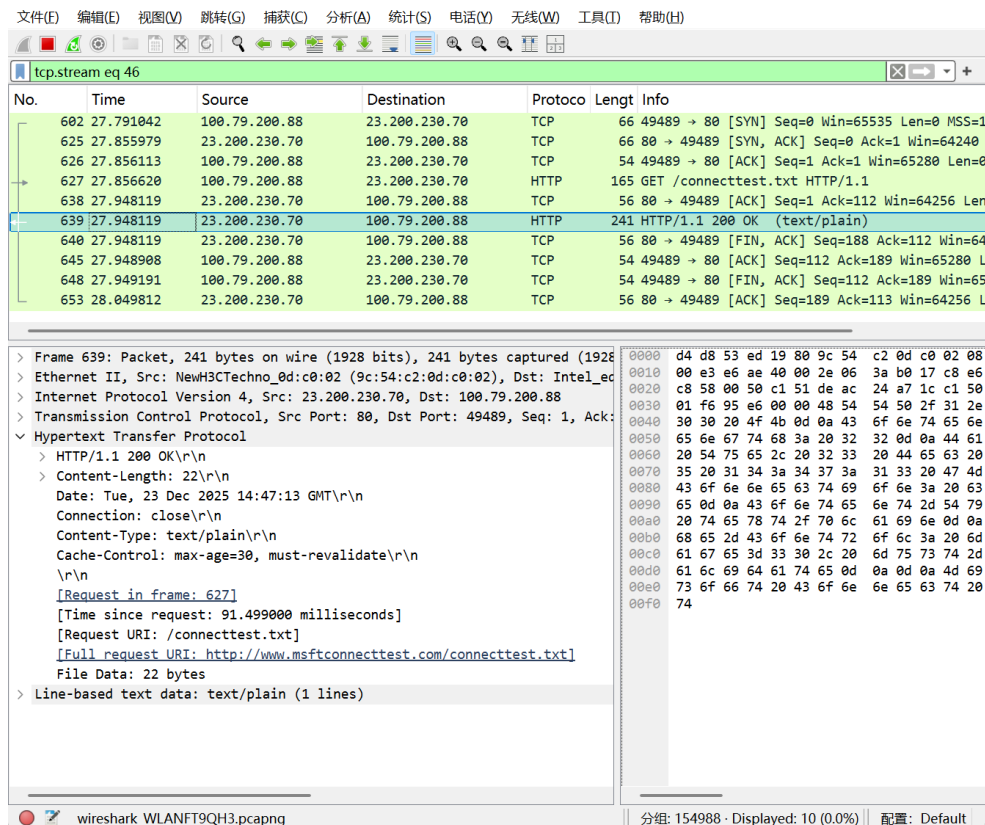
定位应用层数据: 在完成 TCP 三次握手 (Packet 602, 625, 626) 的分析后, 继续在当前的 TCP 流 (Stream 46) 中寻找随后的第一个应用层数据包。

定位 HTTP 请求: 找到列表中的第 627 号数据包, Info 列显示为 GET /connecttest.txt HTTP/1.1, 确认为客户端发起的 HTTP 请求。

定位 HTTP 响应: 找到列表中的第 639 号数据包, Info 列显示为 HTTP/1.1 200 OK (text/plain), 确认为服务器返回的 HTTP 响应。

- **User-Agent: Microsoft NCSI**。它表明发起请求的不是普通的 Web 浏览器（如 Chrome），而是 Windows 系统的网络连接状态指示器。系统通过后台自动访问该文件来判断当前是否已连接到互联网。
- **Connection: Close**。告知服务器在发送完响应后，立即断开 TCP 连接，不保持长连接。

(2) 响应分析



如图所示，数据包 No. 639 是服务器对上述请求的回复。

1. 状态行 (Status Line)

- **内容: HTTP/1.1 200 OK**
- **状态码 (Status Code): 200**。表示客户端的请求已成功被服务器接收、理解并处理。
- **短语 (Phrase): OK**。对状态码的文本描述。

2. 响应头 (Response Headers)

- **Content-Type: text/plain**。告知客户端，返回的数据主体是纯文本格式，而非 HTML 页面。
- **Content-Length: 22**。表示响应实体主体的长度为 22 字节。
- **Date: Tue, 23 Dec 2025...**是服务器生成该响应报文的时间。

3. 实体主体 (Entity Body)

- **内容: Microsoft Connect Test**

这是服务器返回的实际文件内容。Windows 系统在收到这个特定的字符串 "Microsoft Connect Test" 后，就会确认互联网连接正常，并将任务栏右下角的网络图标更新为“已连接”状态。如果内容不匹配或请求失败，系统则会提示“无 Internet 访问”。

八、实验心得与问题分析

8.1. 遇到的问题与解决方案

- 无线 MAC 帧捕获的局限性与解决：

在实验初期，我尝试直接通过 Wireshark 捕获 WLAN 接口数据，试图分析 802.11 帧结构。但我发现，受限于 Windows 操作系统的 NDIS 驱动机制（网卡处于托管模式），捕获到的无线数据包被系统底层自动转换为了伪以太网帧（Ethernet II），导致无法直接观察到 Frame Control、Duration 以及 802.11 特有的三个地址字段。

解决方案：为了完成实验目标，利用 Wireshark 官方提供的标准无线抓包示例文件，成功提取并分析了 Beacon 管理帧、ACK 控制帧和 Data 数据帧，并将其与实际捕获的伪以太网帧进行了对比。

- HTTPS 加密流量的干扰：

在进行 HTTP 协议分析时，我发现访问大多数现代网站（如百度、知乎）抓取到的都是 TLS/SSL 加密数据，无法看到明文的 HTTP 头部。

解决方案：我通过捕捉 Windows 系统自带的网络连接状态检测流量（访问 www.msftconnecttest.com），成功捕获到了未加密的 HTTP 请求与响应。这不仅让我清晰地看到了 GET 请求和 200 OK 响应的结构，还让我了解了操作系统判断网络连通性的工作原理（NCSI）。

- 流量的干扰与筛选：

实验开始时，Wireshark 瞬间捕获了成千上万个数据包，包含了系统后台更新、微信、即时通讯等大量无关流量，导致我很难快速定位到自己发起的测试包。

解决方案：我学会了使用 Wireshark 的过滤器功能。不仅仅是简单的协议过滤（如 http），我还掌握了更精准的过滤条件，例如使用 `ip.addr == 23.200.230.70` 来锁定目标服务器，或者使用追踪流功能屏蔽干扰，只关注当前这条 TCP 连接的完整上下文。

8.2. 实验收获与感悟

本次实验不同于课本上的纯理论分析，我亲眼观察到了数据在协议栈中的封装过程。一个简单的 HTTP GET 请求，在 Wireshark 中被清晰地拆解为应用层（HTTP）、传输层（TCP）、网络层（IP）和链路层（Ethernet/802.11）。这种结构让我对 OSI 七层模型和 TCP/IP 四层模型有了立体、动态的认知，不再局限于书本上的概念。通过对一个完整 HTTP 交互流程的剖析，我直观地看到了数据是如何被层层封装的：用户数据被封装在 HTTP 响应中，HTTP 报文被封装在 TCP 段中，TCP 段被封装在 IP 数据报中，最后被封装在以太网帧中通过物理介质传输。这种层级结构让我对计算机网络体系结构有了更立体的认识。

TCP 协议不再是枯燥的文字描述。通过观察序列号（Seq）和确认号（Ack）的数值变化，我逻辑清晰地推演出了三次握手建立连接、数据传输时的滑动窗口控制、以及四次挥手断开连接的完整生命周期。这种动态的观测让我对 TCP 的“可靠传输”机制有了极强的实感。

此外，我掌握了 Wireshark 的使用。面对海量的网络数据，过滤器（Display Filters）是最高效的工具。通过使用 `wlan.fc.type` 筛选无线帧、使用 `tcp.stream` 追踪完整会话，我从杂乱的背景流量中精准定位到我需要分析的目标数据包，这极大地提升了我的网络分析效率。